Ahoy: LLMs Enacting Multiagent Interaction Protocols

Omkar Joshi[1][0009−0006−5069−2326], Amit K. Chopra[2][0000−0003−4629−7594], and Munindar P. Singh[1][0000−0003−3599−3893]

[1] North Carolina State University, Raleigh, NC, USA
[2] Lancaster University, Lancaster, UK

MPS 1: * refers to my advice pages
MPS 2: G refers to grammar

Abstract. Multiagent systems typically require protocol-specific agent implementations: a change in the protocol necessitates different agents or extensive code modifications. This tight coupling makes it difficult to build flexible agents. We demonstrate that *Large Language Models* can overcome this limitation by enabling a single agent to correctly enact multiple protocols without protocol-specific engineering. We introduce *Ahoy*, an approach that enables LLM-driven agents to select and enact multiple interaction protocols guided by natural language input from users. Ahoy creates agents that are protocol-agnostic — operating without protocol-specific code — while respecting BSPL protocol constraints that ensure correct behavior. Our key contribution is a generic agent adapter (Ahoy) that decouples agent decision making from protocol specification. We evaluate Ahoy across two distinct protocols—Logistics (packaging and shipping coordination) and Purchase (buyer-seller-shipper coordination).

Our results show that Ahoy-enabled LLMs correctly select and enact protocols without specialized training, producing no adapter exceptions, malformed messages, or constraint violations. Ahoy also supports the construction of agents that can intelligently exploit flexible protocols. This approach reduces knowledge engineering overhead by enabling agents to participate in multiple protocols, demonstrating how LLM–enabled multiprotocol architectures enable adaptive coordination in evolving business domains, addressing a key challenge for scaling multiagent systems.

Keywords: Multiagent Systems

1 Introduction

Agentic AI is the idea that .... The promise of agentic AI is ...

Agentic AI is a multiagent enterprise. Agent interaction protocols are therefore crucial. Several protocols have been proposed to support agent interaction, e.g., A2A. They have limitations...

A major limitation of the traditional approach to MAS is that execution of new protocols require the development of new agents.

Recent advances in generative AI have led to the development of Large Language Models that demonstrate reasoning ability.

An LLM agent is ... We investigate the possibility of whether LLM agent may execute information protocols correctly and in a manner that aligns with its user's goals.

Research Question: *Can LLM agents effectively select and execute multiple protocols without protocol-specific training or fine-tuning?*

Our key contributions are as follows:

- A multiprotocol selection framework to detect appropriate protocol based on natural language input.
- A generic agent implementation (Ahoy) that can participate in multiple protocols with minimal code changes.
- Empirical Evaluation across two/three protocols
  OJ 1: subject to time constraint

2 Background

2.1 Multiagent Coordination — Specifying Interaction Protocols

Formally specified rules of encounter, e.g., in BSPL, define the messages, roles, and constraints that guide agent interactions. Traditional approaches require agent implementations to be fixed at design time and changes in the requirement or protocol require agent–level code changes.
BSPL (Blindingly Simple Protocol Language) is a declarative specification language where protocols are described in terms of roles, messages, and logical constraints. OJ 2: add bspl explanation
OJ 3: add bspl protocol example

2.2 Programming Agents Based on Interaction Protocols

Kiko is a framework for implementing protocol-based agents. It provides an adapter that exposes a Python API based on decorators for implementing agents. The API allows developers to plug in the agent's reasoning (e.g. when to take an action or send messages), while the adapter handles prototol compliance.
OJ 4: enhance kiko explanation
OJ 5: insert Kiko example here

2.3 Programming LLM Agents

Large Language Models can be used as the reasoning component in agent architectures. Programming LLM agents typically involves:

- System Prompts
- User Prompts
- Tool Calling
- Memory (RAG/Agent Notes)

These components work together to create an agentic loop where the LLM observes state, reasons about options and takes actions.
OJ 6: Enhance the programming components descriptions
OJ 7: Add a simple example of LLM programming here

3 Architecture

AKC 1: See Azorus for premable The Ahoy framework comprises three layers that take advantage of LLM reasoning to achieve protocol agnosticism. Rather than encoding protocol–specific logic in agents, Ahoy delegates all reasoning to an LLM decision endpoint that understands both the protocol structure and the agent goals. AKC 2: higher layers go higher in the figure

3.1 Three-Layer Architecture

Figure 1 presents the layered design.

Layer 1: Protocol Definition. Interaction protocols are formally specified in BSPL (as described in Section 2). The BSPL specifications are parsed into protocol objects that export message types and roles for convenient access. These protocol objects are augmented with natural language protocol descriptions and sent to the protocol and role selection module.

Layer 2: Protocol Instantiation. An *adapter factory* creates runtime instances of the Kiko Adapter for any protocol–role pair. Given a protocol name and role (e.g. Purchase protocol and the Buyer role), the factory binds the protocol specifications into a concrete adapter instance that tracks social state (message history, parameter bindings) and computes enabled messages — the set of messages that may be legally sent without violating the protocol constraints.

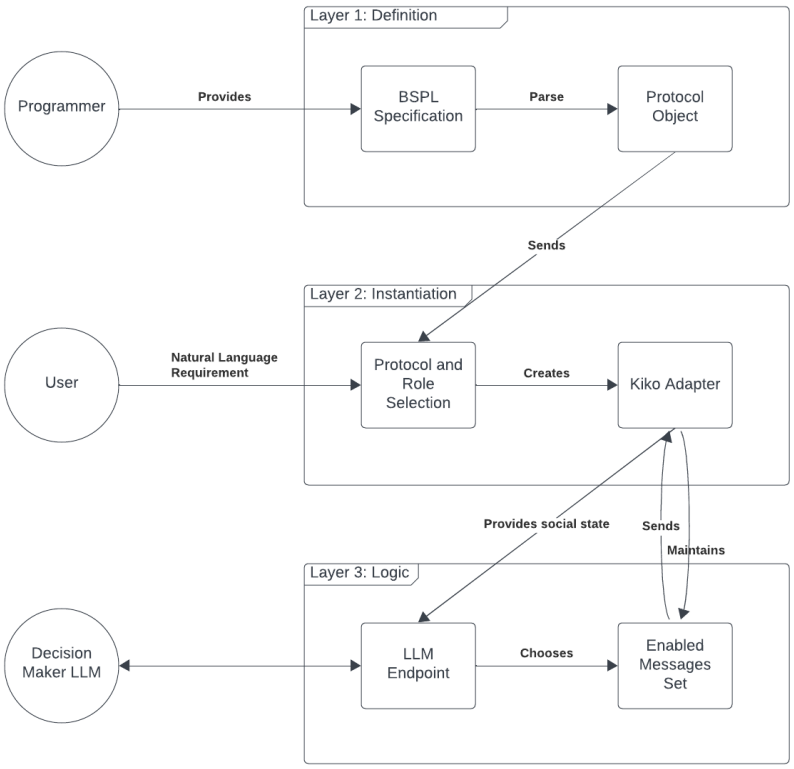

Fig. 1. Ahoy Three Layer Architecture Diagram

Layer 3: Decision Logic. Agents interact with the Kiko adapter with a decision endpoint provided by Ahoy. This is an event handler that observes protocol state changes and delegates reasoning to an LLM. The handler receives the enabled message set and the trigger event and passes that to the LLM along with agent goals and a concise explanation of the BSPL world (Listing 1.1). The LLM reasons about which message(s) to send along with appropriate parameter–value bindings to achieve the user goals. This information is packed into a formatted message instance that the Kiko adapter sends according to the protocol constraints.

Listing 1.1. BSPL Explanation String

```
You are participating in BSPL (Blindingly Simple Protocol
    Language) protocol enactments.
    BSPL defines multi-agent protocols where:
    - Roles are named agents (e.g., Merchant, Buyer)
    - Messages are directed communication between roles
        with parameters marked as 'out' (sender provides)
```

```
    or 'in' (requires prior binding from other
    messages)
- Parameters marked 'key' identify protocol instances
- You play one role and must track received messages
    to determine which messages you can legally send
    next
- A message can only be sent when all its 'in'
    parameters are bound by prior messages
- When multiple messages are enabled, choose based on
     domain reasoning and the protocol's intended flow
```

The adapter enforces protocol–level constraints (message schema, preconditions, ordering) while the LLM handles domain–specific reasoning about optimal actions. This separation is the key to protocol–agnosticism: this approach works across different protocols without modification because Ahoy only requires the LLM to fulfill user goals in an abstract setting with constrained options, a task that LLMs have been shown to perform well.

MPS 3: check every occurrence of "only"

3.2 Guarantees

The adapter enforces two kinds of guarantees. *Structural guarantees* are provided by the Kiko adapter: all messages conform to their schema, and message ordering respects protocol constraints (e.g, an agent cannot send a quote before receiving an rfq if the protocol forbids it). *Logical guarantees* must be provided by agent logic: agents should choose actions that are semantically sound for the domain (e.g, quoting a reasonable price for the requested item). The Ahoy design reflects this separation: the first two layers are responsible for what is *possible*, while the logic layer is responsible for what is *best*.

3.3 LLM Decision Endpoint

The LLM decision endpoint calls a decision handler when a trigger event occurs. This could be a communication event (message received) or a custom event (out–of–stock alert triggered for a product). The first call to the decision handler function provided by Ahoy sets the system prompt. The system prompt contains the information that does not change for all future LLM calls for this specific enactment. This includes information about the BSPL world as shown in Listing 1.1, the agent role the LLM is playing, tool calls available to the LLM, and an explanation of parameter binding rules. This prompt is cached for reuse and overwritten once a new enactment begins. For each call to the LLM, the decision handler function extracts the social state from the Kiko adapter and structures this information into a user prompt. The prompt contains the adapter message history, agent goals, enabled messages, bound parameters, and the parameters that would need to be bound to send each enabled message. Finally, a format enforcement message is added to the end of the prompt, and the LLM is called asynchronously.

4 Programming Model

A key contribution of Ahoy is a programming model centered on the LLM decision endpoint pattern, which enables protocol-agnostic agent implementations. This programming model can be used at three distinct levels of expertise. At the *user* level, end users interact with pre-configured agents and protocols through a dialog interface without writing any code. At the *protocol* level, domain experts can define new protocols and configure agents for these protocols using declarative syntax while writing minimal code. At the *framework* level, we provide support for developers to extend Ahoy by implementing custom event handlers, decision endpoints, or new LLM tooling.

4.1 Level 1: End Users

End users interact with Ahoy agents through the dialog interface. No code knowledge is required. Users can describe their goals in natural language (e.g, "I want to buy a pen and deliver it to [Address]") and Ahoy uses the LLM-based protocol and role selection module to map the user intent to the appropriate protocol and role, then launches the agent automatically. This interface is provided by Ahoy: users do not need to understand protocols, adapters, or LLM endpoints.

4.2 Level 2: Domain Experts

Domain experts can define new protocols and configure agents for them using Ahoy. This level also requires no python coding — declarative BSPL syntax is sufficient. Protocol developers:

1. **Write Protocol Specifications:** Define roles, messages (with typed input and output parameters), and constraints in BSPL (see Section 2). For example, to add a new contract negotiation protocol, a domain expert writes the appropriate BSPL specification defining the Proposer and Acceptor roles, the messages they exchange, and the in/out parameters that govern message ordering.
2. **Register Protocol Metadata:** Add a one–line protocol description to the appropriate file that can be read by Ahoy and serves as an input to the protocol and role selection module.
3. **Configure Agent Behavior:** Specify agent goals and constraints in a human–readable input file (e.g, "Accept offers above $100", "Propose at least a 10% markup"). These constraints are provided to the LLM as part of the system prompt.
4. **Test and Deploy:** Ahoy automatically handles protocol validation, adapter instantiation, and LLM integration. No code changes are required.

In this manner, Ahoy enables domain experts to focus solely on the protocol structure and agent goals, without the need to handle any implementation details.

4.3 Level 3: Framework Developers

Developers can extend Ahoy by adding new tools for LLM use, custom events that can trigger an LLM call, or custom decision strategies that do not need LLM integration. In order to modify Ahoy, developers need to understand the decision endpoint pattern shown in Listing 1.2.

Listing 1.2. Ahoy Decision Endpoint Pattern

```
from bspl.adapter import Adapter
from lib.state_manager import serialize_adapter_state
from lib.llm_client import AnthropicLLMClient

adapter = Adapter(Seller, systems, agents)
llm_client = AnthropicLLMClient()

async def llm_decision(enabled_store, event):
    # Serialize current protocol state
    state = serialize_adapter_state(adapter)

    # Get available messages from enabled_store
    enabled_messages = list(enabled_store.messages())

    # Delegate reasoning to LLM
    llm_response = await llm_client.complete(
        messages=[{
            "role": "user",
            "content": f"State: {state}. "
                       f"Enabled messages: {
                           enabled_messages}. "
                       f"Goal: Choose action (price
                           10-100). "
                       f"What message to send?"
        }]
    )

    # Parse LLM response and bind parameters
    decision = parse_llm_response(llm_response)
    message_instance = bind_message(decision,
        enabled_store)

    return message_instance

# Register handler with decision endpoint
adapter.decision()(llm_decision)
```

5 Demos

Our experiments demonstrate three key capabilities of Ahoy:

1. Protocol Portability: The same agent code executes correctly across different protocols without modification.
2. Enforcement of Guarantees: Structural guarantees are maintained in all enactments.
3. Protocol Selection Accuracy: The protocol selection module reliably maps user intent to appropriate protocols and roles.

5.1 Demo 1: Protocol Portability

Goal: Demonstrate that a single Ahoy agent can enact multiple protocols without code changes.
Method: Execute identical agent decision logic in two distinct protocol contexts:

- Purchase Protocol (Buyer role): Agent participates in a buyer–seller-shipper negotiation.
- Logistics Protocol (Merchant role): Agent coordinates packaging and shipping workflow.

Both enactments use the same underlying LLM decision endpoint pattern shown in Listing 1.2. No protocol–specific logical branches or conditional code paths are used.
Results: We capture execution traces from both protocols and verify the following.

- Both protocols reach terminal state.
- Message sequences respect protocol–specific constraints.
- No adapter exceptions or schema violations occur.
- LLM decision quality is consistent across both domains. OJ 8: no stupid decisions

5.2 Demo 2: Guarantee Validation

Goal: Verify that the framework guarantees hold across protocols.
Method: We execute curated scenarios targeting the core guarantees:

- Message Validity: Verify that only schema–conforming messages are offered to the LLM.
- Parameter Scoping: Test scenarios where identical parameter names (e.g. orderID) appear in two or more protocols; verify that they remain isolated and do not contaminate across protocol contexts.
- Role Consistency: Confirm that only role–appropriate messages appear in the enabled message set at each decision point.

Results: We produce a trace validation report documenting:

- Scenario description
- What would break in a naive agent
- How Ahoy handles it.
- Exceptions noted.

5.3 Demo 3: Concurrent Multiprotocol Participation

Goal: Demonstrate that a single LLM-driven agent can simultaneously participate in multiple protocol instances while maintaining state isolation and making coherent decisions across different protocol contexts.
Method: We instantiate parallel protocol enactments where the agent operates in distinct roles across multiple protocols concurrently:

- **Scenario Setup:** Agent acts as Buyer in Purchase protocol while simultaneously acting as Merchant in Logistics protocol.
- **State Isolation:** Each protocol maintains an independent adapter with isolated social state (message history, parameter bindings, enabled messages).
- **Decision Interleaving:** We trigger decision events across both protocols in an interleaved fashion, simulating realistic concurrent coordination.
- **Context Switching:** The LLM receives distinct user prompts for each protocol, including protocol-specific enabled messages and agent goals, testing whether the LLM maintains context separation.

Results: We capture execution traces from both concurrent protocol enactments and verify:

- **Parameter Isolation:** Parameters bound in one protocol (e.g., orderID in Purchase) do not interfere with identically-named parameters in another protocol (e.g., orderID in Logistics).
- **Message History Separation:** Each protocol's message history remains isolated; the LLM references only relevant messages when making decisions in each context.
- **Coherent Decision Making:** Decisions made in one protocol do not violate constraints or create contradictions in the other protocol.
- **No Cross-Protocol Contamination:** Enabled message sets correctly reflect only messages available in each specific protocol context.
- **Concurrent Termination:** Both protocols reach terminal state independently without deadlock or state corruption.

5.4 Demo 4: Decision Quality Across Domains

Goal: Show that the LLM reasoning adapts seamlessly and makes semantically appropriate choices in different protocol contexts.
Method: We capture decision point snapshots from representative protocol enactments:

- Purchase Protocol: LLM chooses the appropriate price to buy items.
- Logistics Protocol: LLM chooses the appropriate material to wrap the package based on frailty without explicit guidance.

Results: Table showing

- Scenario description
- LLM action and reasoning
- Semantic appropriateness of decision in context (LLM as judge?)

5.5 Demo 5: Protocol Selection Accuracy

Goal: Validate that the protocol selection module reliably maps user intent to correct protocol–role pairs.
Method: We construct a test set of n user intent statements, including:

- Clear Purchase Intents: I want to buy a notebook
- Clear Logistics Intents: I want to organize packaging and shipping for a warehouse containing the following items ...
- Ambiguous intent: I want to ship something

Each test case has a ground truth protocol label.
Results: A results table showing

- User input
- Ground truth label
- Difficulty marker
- Selection output

and a summary table showing the accuracy, recall and precision.

5.6 Demo 6: Custom LLM Events

Goal: Demonstrate that Ahoy agents can respond to both protocol-driven events (received messages) and custom business logic events (timeouts, thresholds, periodic checks) using the same LLM decision endpoint.
Method: We instantiate agents that handle concurrent event sources:

- **Adapter Reactions:** Traditional protocol event handlers triggered when messages arrive.
- **Custom Events:** Business logic triggers such as periodic timeout checks, stall detection, or out-of-stock alerts.
- **Synchronization:** Both event paths converge on the same `choose_and_bind()` decision logic, synchronized via `asyncio.Lock` to prevent concurrent access conflicts.

Test scenarios include:

- Purchase Protocol (Buyer with periodic timeout): Adapter processes incoming quotes; custom event handler checks every 2 seconds whether to send follow-up messages.
- Logistics Protocol (Merchant with stall detection): Adapter processes shipping requests; custom event handler detects prolonged inactivity and triggers proactive decisions.

Results: We verify that:

- **Concurrent Execution:** Both adapter and custom event paths execute without race conditions or deadlocks.

- **Unified Decision Logic:** Both event types use identical parameter-binding and message-sending workflows.
- **Constraint Preservation:** Messages from both paths respect protocol constraints and parameter bindings.
- **Metric Tracking:** LLM call counts correctly reflect decisions from both adapter and custom event handlers.
- **Graceful Termination:** Agents exit cleanly when protocols complete or thresholds are exceeded.

5.7 Summary

Together, these experiments demonstrate that Ahoy achieves protocol–agnosticism. A single, generic agent implementation using Ahoy, correctly selects and enacts multiple protocols, fulfills all structural guarantees, and makes semantically sound decisions without protocol–specific code modifications.

6 Related Work

This section positions our work within the broader landscape of multiagent systems, LLMs and agent programming models.

6.1 LLMs in Multiagent Systems

Recent work explores the use of Large Language Models as reasoning engines in multiagent systems. Key approaches for LLM interaction include:

- In Context Learning
- Fine Tuning
- Chain of Thought Reasoning

In addition, there has been a push towards improving communication between LLM agents. Various approaches define protocols and communication patterns that enable LLM-based agents to coordinate effectively. Our work differs from existing approaches by focusing on *dynamic protocol selection and enactment:* given user intent expressed in natural language dialog, our framework automatically selects from multiple available protocols and binds the agent to the appropriate role without requiring protocol–specific agent implementations. OJ 9: Explain how we differ from each prompting approach

6.2 Programming Models

Established programming models for multiagent systems include the Actor model, reactive agents and BDI (Belief–Desire–Intention) architectures. These provide different abstractions for agent behavior and coordination. Our Ahoy framework complements these by ...

[**?**]